

Développement de l'informatique embarquée du projet WAQATALI

Pierre BARBAT

Du 20 Février 2023 au 30 Juin 2023

Responsables de l'entreprise : Nicolas MARILLEAU & Loïc HALLEZ
Responsable de l'université : Alain GIORGETTI



Sommaire

1	Glossaire	5
2	Remerciements	6
3	Introduction	7
4	Contexte	8
5	Environnement de travail	10
5.1	L'UMMISCO de l'IRD	10
5.2	L'institut UTINAM de l'Université de Franche-Comté	11
5.3	UrbaSense	11
6	La station QameleO	12
6.1	La carte mère	12
6.2	Les micro-capteurs	13
6.3	Les modules de traitement des données du QameleO	15
6.4	Workflow des stations QameleO	17
6.5	Le travail effectué	18
6.5.1	Modification matériel et logiciel du module GSM pour fiabiliser la connexion au serveur	18
6.5.2	Communication avec le module SD et validité des données	19
6.5.3	Améliorer l'ergonomie du paramétrage de QameleO	20
7	Le capteur Waou	21
7.1	Le fonctionnement	21
7.2	Le capteur à mon arrivée	22
7.3	Développement du dispositif de mesure expérimental Waou	23
7.3.1	Phase 1 : Suppression des relais	23
7.3.2	Phase 2 : Les composants et branchements de la carte Waou	25
7.3.3	Phase 3 : Les limites fréquentielles d'une carte Arduino . .	28
7.4	Réglage de la fréquence de mesure	31
7.4.1	Méthodes	31
7.4.2	Résultat	32
7.5	Le capteur aujourd'hui	32
7.5.1	Résultats	32
7.5.2	Mise en place d'un protocole pour trouver la source de l'erreur	34
8	Conclusion	36
8.1	Conclusion techniques	36
8.1.1	QameleO	36
8.1.2	Waou	36
8.2	Conclusion personnel	36

9 Bibliographie	37
10 Annexes	38
11 Résumé / Abstract	43

Liste des figures

1	Ensemble des organismes travaillant sur le projet WAQATALI . .	10
2	Station QameleO	12
3	Shield de la station QameleO	12
4	Carte Arduino Mega 2560	13
5	Capteur Tera - Next PM	14
6	Capteur d'humidité et température : SHT35	14
7	Capteur "Air Quality 3 click"	15
8	Module SD HW-125	15
9	Module GSM SIM800L	16
10	Workflow des QameleO	17
11	Schéma branchement normal en série et branchement QameleO .	18
12	Code de la récupération des données de la carte SD	19
13	Code de l'envoi des données	19
14	Extrait du fichier de configuration	20
15	Schéma du montage pour réaliser une mesure	21
16	Photo de la boîte à "clic-clac"	22
17	Photo de la sonde Watermark	22
18	Montage électrique pour tester l'impédance	24
19	Capture d'écran des résultats de la mesure de résistance avec tout les pins en OUTPUT	24
20	Capture d'écran des résultats de la mesure de résistance avec les pins en OUTPUT - INPUT	25
21	Module RTC	25
22	Schéma de la configuration de la carte électronique	26
23	Photo de la carte Waou	26
24	Photo de la carte Waou soudé	27
25	Prototype complet Waou	27
26	Ligne de réglage du code Waou	28
27	Code du passage de 0V à 5V avec digitalWrite()	28
28	Code du passage de 0V à 5V en modifiant l'état des pins	29
29	Code de l'ajout d'un delay arbitraire	29
30	Code de l'ajout d'un delay calculé	30
31	Résultat des mesures du capteur 1	33
32	Résultat des mesures du capteur 3	33
33	Résultat des mesures du capteur 5	34
34	Graphique d'évolution de la résistance en fonction de la fréquence de mesure	35
35	Mesure de la méthode "digitalWrite"	38
36	Mesure de la méthode "porth" sans délai	39
37	Mesure de la méthode "porth" avec délai par défaut	40
38	Mesure de la méthode "porth" avec délai réglé pour 20 kHz . . .	41
39	Mesure de la méthode "porth" avec délai réglé pour 500 kHz . . .	42

1 Glossaire

- WAQATALI : WAter & Air QuAlity monitoring for Smart VegeTAL Infrastructures in cities
- QameleO : Quality of Air Module for Environmental Learning Engeenering and Observation
- Waou : Water Available Optimum Usage
- Résistance : l'aptitude d'un matériau conducteur à s'opposer au passage d'un courant électrique sous une tension électrique donnée
- GBF : générateur de basses fréquences
- SDA : Serial Data Line
- SCL : Serial Clock Line
- Micro-capteur : Un micro-capteur est un composant miniaturisé dédié à la mesure (ou la représentation de la mesure) de composés présents dans l'air. Il est basé sur un phénomène physique, chimique ou biologique permettant de transformer une grandeur physico-chimique en un signal, généralement électrique

2 Remerciements

Je tiens à remercier mes maîtres de stage Mr. Nicolas Marilleau et Mr. Loïc Hallez pour leur aide tout au long de mon stage et pour mon accueil au sein du laboratoire UTINAM. Je remercie également Mme. Manel Rami avec qui j'ai travaillé sur le développement du code et de la carte électronique pour le capteur Waou et Paul Gruet-Masson, stagiaire en 2ème année de BUT Chimie des matériaux qui a travaillé sur les matériaux du capteur Waou. Je remercie aussi mon tuteur de stage Mr. Alain Giorgetti qui a pris le temps de venir me voir et m'a aidé dans mon rapport de stage.

Merci également à toute l'équipe SRS de l'institut UTINAM pour l'accueil chaleureux et notamment à Vanessa Avramovic qui a pris le temps de m'apprendre les bases de l'électronique et de la soudure.

3 Introduction

L'informatique embarquée est de plus en plus répandue dans le monde, que ce soit dans des objets simples comme des ampoules ou dans des objets beaucoup plus complexes comme des fusées ou voitures. Toutes ces technologies nous font réaliser l'importance des capteurs car ils permettent d'augmenter la sécurité pour un usager (le système d'airbag dans une voiture), d'améliorer le confort (Climatisation automatique) ou encore dans la prise de décision grâce à des données mesurées par ces capteurs. C'est avec la volonté d'apprendre à utiliser ces capteurs que j'ai réalisé un stage financé par l'IRD (Institut de Recherche pour le développement) dans le cadre du projet WAQATALI. Ce stage est dans le cadre de mon second semestre de deuxième année de Master informatique à l'Université de Franche-Comté et il s'est déroulé du 20 Février 2023 au le 30 Juin 2023 à l'institut UTINAM de Besançon sur le site de l'IUT de Chimie.

Ce rapport présentera en premier lieu le projet WAQATALI dans sa globalité puis les partenaires universitaires tels que l'IRD et l'institut UTINAM et les partenaires industriels comme UrbaSense. Je détaillerai ensuite le travail que j'ai réalisé durant ces 18 semaines de stage sur l'informatique embarquée du capteur Waou, l'écriture de documentation et la correction d'erreur concernant la transmission des données. Je terminerai ce rapport en parlant des perspectives d'évolution du projet et des compétences apprises lors de ce stage.

4 Contexte

Les villes et les sociétés actuelles connaissent de profondes mutations pour faire face aux enjeux climatiques, environnementaux et sociétaux qui sont prégnants au Nord et au Sud. C'est pourquoi le projet WAQATALI (WATER & AIR QUALITY monitoring for Smart VEGETAL Infrastructures in cities) tente de répondre à ces questions. Il s'agit d'un laboratoire commun financé par l'ANR (l'Agence Nationale de la Recherche) pour un budget de 362 999 € et sur une durée de 5 ans. Ce projet se base sur un partenariat entre l'entreprise UrbaSense et l'Unité Mixte Internationale UMMISCO (Unité de Modélisation Mathématique et Informatique des Systèmes Complexes) de l'IRD (Institut de Recherche pour le Développement). D'autres laboratoires interviennent dans ce projet en qualité d'expert. L'ensemble des recherches sur ce projet permettra de produire des données environnementales, de développer une suite cohérente d'instrumentations, d'outils de traitements de données, d'indicateurs et de méthodologies d'appui à la décision, dont l'utilisation s'inscrira dans les missions scientifiques et commerciales. Deux capteurs, QameleO et Waou vont mesurer des paramètres environnementaux (température, qualité de l'air, l'humidité de surface...). La station de mesure environnementale QameleO (Quality of Air Module for Environmental Learning Engineering and Observation) a pour but de mesurer le taux de particules dans l'air à coût réduit partout dans le monde. Le code logiciel, les plans de conception (schéma électronique, modèle 3D, etc), et les manuels de constructions sont respectivement sous license GPL3, CERN-OHLV2, et creative common, à savoir des licenses Open-Source. Ainsi, cette station est accessible gratuitement en ligne et modifiable à la convenance de l'utilisateur notamment pour un usage en FABLAB. Quant à la station Waou (Water Available Optimum Usage), elle sera développée dans le but de mesurer le besoin en eau des végétaux et également à coût réduit partout dans le monde. Tout comme QameleO, tout sera Open-Source. Ces mesures seront ensuite utilisées pour créer un service de concertation et de planification participative pour mettre en place des aménagements verts.

A mon arrivé, le capteur QameleO était déjà développé mais des bugs logicielles étaient présents. Quant à Waou, le capteur était au début de sa conception : il n'existait qu'un prototype permettant de faire une mesure pour un seul capteur sans fiabilité assurée et sans réglage possible.

Durant ce stage, j'ai contribué activement à :

- la station environnementale QameleO en corrigeant des bugs, en ajoutant un micro-capteur.
- la conception et l'implémentation de l'informatique embarquée du capteur Waou.

La station Waou devra être capable de mesurer une tension et de calculer la résistance du capteur Waou et de sauvegarder ces valeurs sur une carte SD.

Suivant la valeur de cette résistance, on va pouvoir définir l'humidité de celui-ci. La station QameleO, étant déjà développée, m'a servi à apprendre l'informatique embarquée. J'ai donc débuté avec celle-ci avant de pouvoir corriger ces bugs comme le module de carte SD ne se connectant pas et des erreurs lors de l'envoi des données via un GSM et rédiger un guide d'utilisation. Le stage s'est déroulé à l'IUT de Chimie de Besançon où les matériaux utilisés pour le capteur Waou sont créés, testés et choisis par Paul Gruet-Masson, stagiaire en 2ème année de BUT Chimie. Cela a permis de tester le code ainsi que le capteur sur place.

5 Environnement de travail

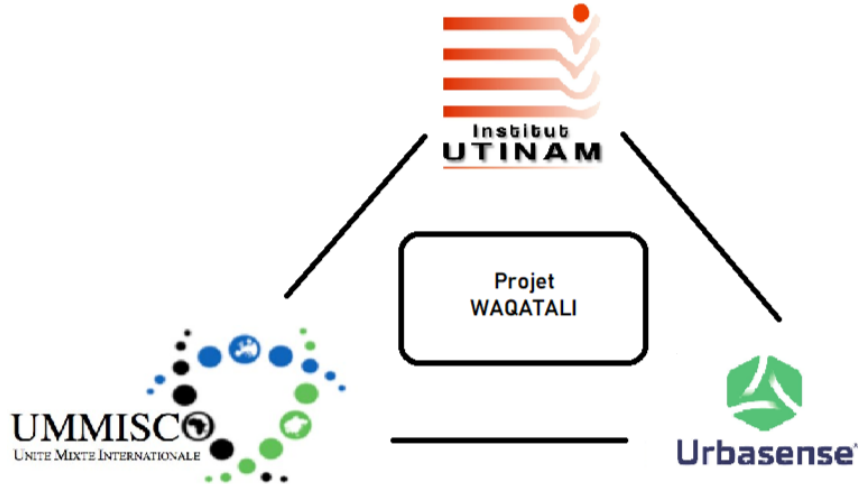


Figure 1: Ensemble des organismes travaillant sur le projet WAQATALI

5.1 L'UMMISCO de l'IRD

L'IRD est un organisme de recherche français, un EPST (établissement public à caractère scientifique et technologique) reconnu internationalement et travaillant avec les pays du Sud et les îles d'Outre-Mer française. Son travail consiste en la réalisation des Objectifs de Développement Durable (ODD) et de proposer, avec des partenaires, des solutions concrètes sur des problèmes Sociétal et planétaire.

L'institut a été créé en 1944 et son siège social est situé à Marseille. 2186 fonctionnaire de l'Etat français travaillent à l'IRD dont 26% d'entre eux sont affectés hors métropole sans compter les contrats ou les agents locaux, les doctorants et les post-doctorants répartie dans 80 unités et services dédiées à la recherche. Pour organiser les collaborations, l'IRD compte sur son réseau de 38 représentants. Grâce à ces représentants, l'institut est présent dans une cinquantaine de pays et participe dans la gestion des crises environnementales, sanitaires ou également dans les dispositifs de prévention et préparation nécessitant une expertise pluridisciplinaire.

L'Unité Mixte International UMMISCO (Unité de Modélisation Mathématique et Informatique des Systèmes Complexes) apporte une expertise sur l'informatique du projet WAQATALI. Cette Unité composée de 76 membres en 2018 est sous la tutelle de 7 organismes universitaires en France (IRD, Sorbone Université), au Maroc (UCA - Université Cadi Ayaad), au Sénégal (UCAD - Université Cheick Anta Diop , UGB - Université Gaston Berger), au Cameroun (Université Yaoundé 1) et au Vietnam (USTH - Université des Sciences et Technologies). Les thématiques scientifiques d'UMMISCO sont dédiées à la conception d'approches et d'outils de modélisation mathématique et informatique qui permettent de

mieux appréhender et comprendre les dynamiques émergentes des "systèmes complexes", de dessiner leurs scénarios d'évolution les plus probables, et d'alimenter la prise de décision en matière de développement durable.

5.2 L'institut UTINAM de l'Université de Franche-Comté

L'institut UTINAM (Univers, Temps-fréquence, Interfaces, Nanostructures, Atmosphère et environnement, Molécules) est une unité de recherche pluridisciplinaire placée sous la tutelle du CNRS et de l'Université de Franche-Comté.

L'institut UTINAM est composé de 5 équipes pluridisciplinaires dont l'équipe SRS (Sonochimie et Réactivité des Surfaces) qui m'a accueilli dans ses locaux. Cette équipe travaille sur les traitements de surface et plus généralement sur les matériaux.

5.3 UrbaSense

UrbaSense est une entreprise fondée en 2015 et située à Versailles. L'ambition de l'entreprise est de végétaliser les villes efficacement tout en étant économe sur l'utilisation des ressources telles que l'eau. Pour cela, elle propose un suivi des espaces de nature en ville sur l'ensemble de leur cycle de vie et de donner des recommandations adaptées et personnalisées aux utilisateurs.

Ces différents groupes travaillent ensemble sur le projet WAQATALI car chacun d'entre eux possède sa spécialité. L'UMMISCO apporte des connaissances sur l'informatique et des pédologues, UrbaSense est expert en végétalisation des villes et des capteurs nécessaires à ce processus et l'institut UTINAM donne son expertise sur les matériaux.

6 La station QameleO

Le but de la station QameleO est de mesurer les particules fines dans l'air ainsi que l'humidité et la température. Les mesures sont ensuite enregistrées et envoyées sur un serveur distant. Le code et toutes les consignes de montage du boîtier sont disponible Open-source sur le lien suivant : <https://github.com/CoFab-in-Bondy/QameleO>. La station est fabriquée en FABLAB et elle permet d'être assemblée dans des collèges et lycées afin de faire découvrir l'électronique et l'informatique à des néophytes. Ils vont donc pouvoir assembler la carte mère du QameleO avec l'ensemble des composants de la station.

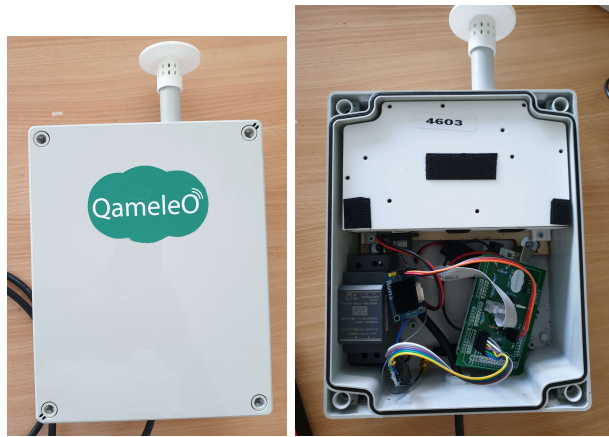


Figure 2: Station QameleO

6.1 La carte mère

La carte mère est le circuit électronique comprenant tout les composants pour brancher d'une part les micros capteurs, les modules de traitement des données ainsi que le processeur Arduino. Dans la communauté Arduino, cette carte mère s'apparente à ce qu'on appelle un shield (voir figure ci-dessous).

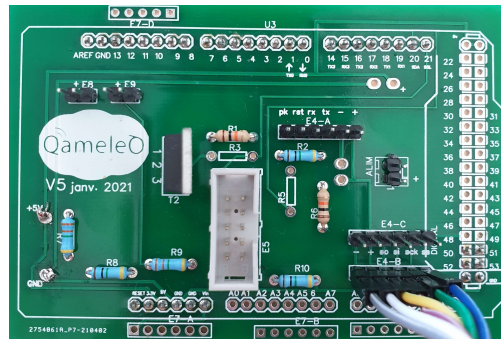


Figure 3: Shield de la station QameleO

Ainsi, les développements d'informatique embarquée de QameleO se repose sur des bibliothèques et du code adaptées au processeur incluses dans les cartes Arduino (voir figure ci-dessous). Cette carte est l'équivalent d'un micro-ordinateur. C'est à dire que la carte est capable d'effectuer des opérations, de stocker des variables, etc. Néanmoins, il faut faire attention car la mémoire disponible et la puissance de calcul sont très limités.

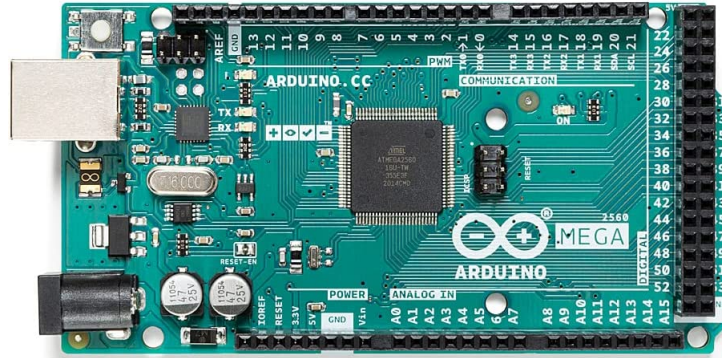


Figure 4: Carte Arduino Mega 2560

Cette carte va donc servir à faire des mesures, des calculs, enregistrer des données et les envoyées sur un serveur distant et contrôler d'autres composants. Les pins numérotés de 0 à 53 sont des pins digitaux. Ces pins peuvent être paramétré en tant que **OUTPUT**. C'est à dire qu'on va pouvoir envoyer une tension à partir de ces pins et alimenter d'autres composants. On peut également les mettre en mode **INPUT**. Une fois ce mode sélectionné, ces pins détecteront tout signal électrique renvoyé par un capteur comme un bouton par exemple. Les pins analogiques, numérotés de A0 à A15, vont pouvoir lire une tension. Arduino est une carte facile d'apprentissage et d'utilisation. Son principal avantage est la quantité/qualité de la documentation sur Arduino et son IDE qui permet d'envoyer son code dans la carte simplement. En plus de la facilité d'utilisation de l'Arduino, plusieurs protocoles de communications sont implémentés par défaut comme le protocole I2C, SPI ou encore en Série permettant ainsi la communication entre les composants facile à programmer.

6.2 Les micro-capteurs

Dans la station QameleO, le composant principal est un capteur permettant de mesurer les particules dans l'air appelé "Next PM" utilisant le protocole UART et produit par la société "Tera" (voir figure ci-dessous).

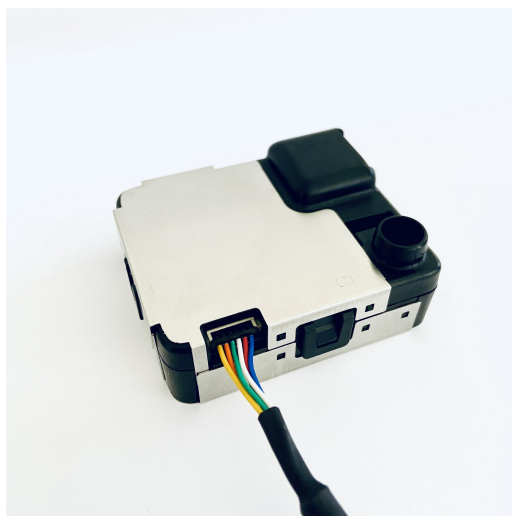


Figure 5: Capteur Tera - Next PM

Ce capteur va donc mesurer plusieurs catégories de particules dans l'air : Les PM1, les PM2.5 et les PM10. Pour éviter tout risque de surchauffe, ce capteur possède un appareil de mesure de température et d'humidité intégré.

Afin de pouvoir mesurer la température extérieur et l'humidité, QameleO est équipée d'un capteur SHT35 avec lequel on communique via le protocole I2C (voir figure ci-dessous).

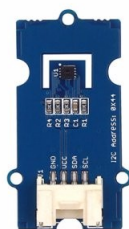


Figure 6: Capteur d'humidité et température : SHT35

En plus de ces deux micro-capteurs, il est également possible d'installer de nouveau capteur sur le QameleO. J'ai travaillé sur le capteur "Air Quality 3 click" (voir figure ci-dessous) afin d'essayer de le rendre compatible à notre programme en utilisant le protocole I2C.

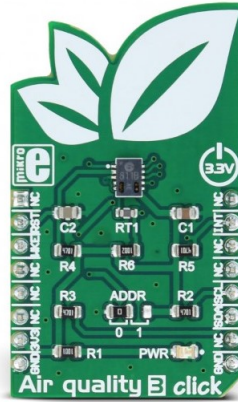


Figure 7: Capteur "Air Quality 3 click"

Ce micro-capteur est capable de nous aider à estimer la qualité de l'air en mesurant la quantité de CO₂ et la concentration de composant organique volatile (CoV) présent dans l'air. Actuellement, le code n'est pas fonctionnelle mais des tests sur le capteur en lui-même sont en cours.

6.3 Les modules de traitement des données du QameleO

Une fois que les mesures sont faites, elles sont enregistrées sur une carte SD. Un module SD HW-125 (voir figure ci-dessous) permettant de lire et écrire des données à donc été installé sur la carte Arduino. Toute la communication avec ce module utilise le protocole SPI.



Figure 8: Module SD HW-125

Et enfin, pour envoyer et recevoir les données d'un serveur distant, on va utiliser un module GSM SIM800L (voir figure ci-dessous). C'est un boîtier électronique qui va se connecter au réseau téléphonique grâce à une carte SIM.



Figure 9: Module GSM SIM800L

On va donc pouvoir utiliser le **protocole MQTT** et communiquer entre équipement et serveur. Le protocole MQTT (Message Queuing Telemetry Transport) est un protocole Open-Source qui assure des communications non permanentes entre des appareils par le transport de leur message.

Un autre point important est la communication entre les composants et la carte Arduino. C'est pourquoi chaque composant est compatible avec le **protocole de communication I2C, SPI ou Serial**. I2C (Inter-Integrated Circuit) est un bus série bidirectionnel où plusieurs équipements, maître ou esclave peuvent se connecter à ce bus. La communication s'établit toujours entre un maître et un ou plusieurs esclaves. Donc dans notre cas, le maître serait la carte Arduino et les esclaves, les différents composants cités ci-dessus.

La carte Arduino possède des pins SDA et SCL, tout comme les composants compatibles en I2C. Le pin SDA va servir à transmettre les données et le pin SCL à transmettre une horloge de synchronisation. A chaque "coup" d'horloge, un bit est envoyé. Pour cela, il est important que les pins SDA soient branchées entre eux et les pins SCL soient également branchés entre eux. Le fonctionnement est le suivant :

- Le maître crée une trame avec l'adresse de l'esclave cible et la commande que l'esclave doit exécuter.
- Seul l'esclave cible reçoit la trame et exécute la commande.
- L'esclave envoie dans une nouvelle trame le résultat de la commande.
- Le maître reçoit les données et peut les exploiter.
- NOTE IMPORTANTE : le format de la trame dépend du composant avec lequel on souhaite interagir. Cette information se trouve dans la fiche technique du composant.

Le protocole de communication SPI est utilisé par notre module SD. Une transmission SPI typique est une communication simultanée entre un maître et un esclave :

- Le maître génère l'horloge et sélectionne l'esclave avec qui il veut communiquer par l'utilisation du signal SS
- L'esclave répond aux requêtes du maître

Contrairement au protocole I2C, on n'utilise pas les pins SDA et SCL mais on utilise les 4 pins suivants :

- MISO (Master in Slave Out) est la ligne pour envoyer des données au maître;
- MOSI (Master Out Slave In) est la ligne pour envoyer des données à l'esclave;
- CLK (Serial Clock) est une ligne de synchronisation des appareils générée par le maître;
- SS (Slave Select) – ligne permettant d'activer/désactiver les dispositifs esclaves.

Dans le cas de ce protocole, il faut brancher le pin MISO avec le pin MOSI du composant et le pin MOSI avec le pin MISO. Il faut donc croiser les branchements.

À chaque coup d'horloge le maître et l'esclave s'échangent un bit. Après huit coups d'horloges le maître a transmis un octet à l'esclave et vice versa. La vitesse de l'horloge est réglée selon des caractéristiques propres aux périphériques.

6.4 Workflow des stations QameleO

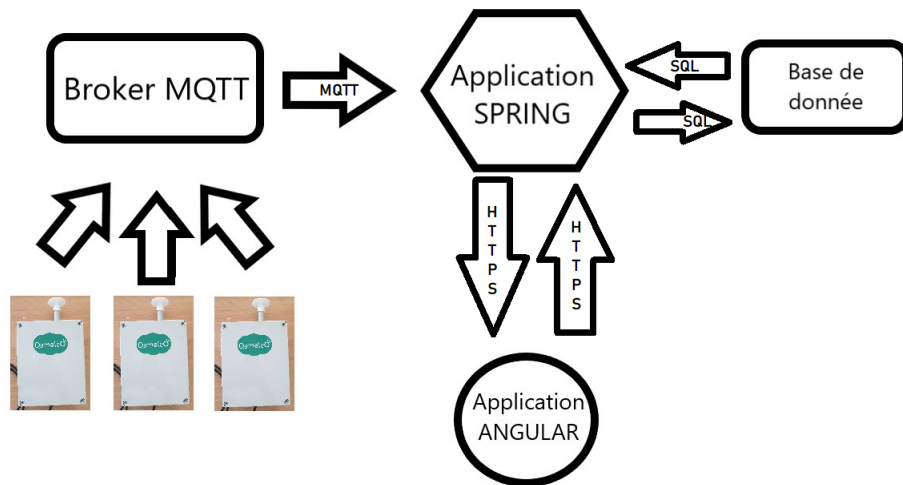


Figure 10: Workflow des QameleO

Le workflow des QameleO se fera comme sur la figure ci-dessus. Pour commencer, les stations QameleO vont faire leurs mesures. Une fois que les mesures sont faites, elles sont envoyées sur un broker MQTT. Une application SPRING va venir lire dans ce broker les trames et va les traiter pour ensuite enregistrer les mesures dans une base de donnée. L'utilisateur peut ensuite utiliser une application ANGULAR pour voir ses données sur son téléphone ou ordinateur.

6.5 Le travail effectué

6.5.1 Modification matériel et logiciel du module GSM pour fiabiliser la connexion au serveur

La station QameleO fournie avait des problèmes sur l'envoi des données. j'ai donc dû chercher la cause dans le code et dans les branchements des différents composants. Après avoir téléchargé le code, j'ai commencé à écrire un tutoriel d'installation et de paramétrage afin d'éviter toute erreur de compilation. Ce tutoriel est disponible dans l'archive du projet QameleO. En démarrant la station, on constate que effectivement que le GSM, responsable de l'envoi des données n'arrivait pas à se lancer. Cependant, en lisant la documentation Arduino, le code n'avait pas d'erreur. J'ai donc regardé les branchements en suivant les instructions du constructeur du GSM. Après vérification, trois erreurs ont été trouvées. La première et plus évidente est le manque d'une carte SIM ce qui rend la connexion impossible. La deuxième raison est la façon dont était connecté le GSM. Ce GSM possède quatre pins. Deux pour l'alimentation et deux pour la transmission des données avec le protocole Serial entre la carte Arduino et le GSM appelés Rx et Tx. Le pin Rx (Receiver) va servir pour la réception de la donnée et le pin Tx (Transmitter) va servir à l'envoi de la donnée.



Figure 11: Schéma branchement normal en série et branchement QameleO

Normalement, à l'instar du protocole SPI, le protocole Serial se branche en croisé comme le montage gauche de la figure ci-dessus. C'est à dire que le pin Rx de l'Arduino se branche sur le pin Tx du GSM et que le pin Tx de l'Arduino se branche sur le pin Rx du GSM. Cependant, la carte électronique QameleO a été conçue pour être facile d'utilisation et pour être facile de montage. C'est pourquoi le branchement souhaité se voulait être le plus intuitif possible en faisant comme sur la partie droite du schéma ci-dessus. La troisième erreur est l'alimentation. Lors de mes tests, j'ai branché la station QameleO à mon PC. Cependant, la tension fournie par mon PC n'est pas suffisante et l'alimentation

du GSM était trop faible pour fonctionner. J'ai donc branché en plus la station QameleO sur secteur en plus de mon PC et, la combinaison des trois corrections ont permis au GSM de se connecter.

6.5.2 Communication avec le module SD et validité des données

Une fois que la connexion est établie, j'ai constaté qu'un autre problème était apparu car les données n'étaient jamais envoyées. En lisant en détail le code ci-dessous, on remarque que les données à envoyer sont récupérées dans la carte SD. On peut en déduire que l'interaction avec la carte SD ne se fait pas et que les données n'arrivent pas à être récupérées.

```
Queue<String> dataToSend = mySD.getDataToSend();  
if(dataToSend.isEmpty())  
    Serial.println("There are no data to send");
```

Figure 12: Code de la récupération des données de la carte SD

J'ai donc vérifié les branchements du lecteur de carte SD et constaté qu'ils n'étaient pas bon. Une fois les branchements corrigés, l'envoi s'est fait. Après vérification de la validité des données, on constate que la date de chaque mesure est incorrecte car la date est en Unix Time mais arrive en étant négative.

```
getNbReboot());  
if (myGSM.sendData(myDataMsg.haveUnixTime(tmpString,timerSystem.  
getNbReboot()), dataToSend.isEmpty())){  
    Serial.println("The data has been send.");  
} else {  
    Serial.println("Failed, the data hasn't been send.");  
}
```

Figure 13: Code de l'envoi des données

Le code ci-dessus montre que le GSM possède une fonction **sendData()** et la fonction **haveUnixTime()**. Cette dernière est supposée modifier la date de la mesure pour mettre la véritable date à la seconde près car la date inscrite par défaut est le temps en milliseconde écoulé depuis le lancement de la station. La station QameleO peut récupérer la date en UnixTime sur un serveur. Cependant, cette opération est énergivore. C'est pourquoi elle est récupérée toute les quatres heures dans le réglage par défaut. Grâce à ça, on va pouvoir calculer précisément la date en fonction de l'heure inscrit dans le message par défaut, l'heure envoyé par le serveur et le temps écoulé dans la machine.

6.5.3 Améliorer l'ergonomie du paramétrage de QameleO

Afin de rendre la station environnementale QameleO accessible à tous, un fichier de configuration a été créé. L'utilisateur va pouvoir modifier directement les valeurs dans ce fichier (voir figure ci-dessous).

```
#define SENSOR_NAME "azerty"

//MEASURE TIME AND SEND DATA TIME
#define MEASURE_PERIOD 15
#define SEND_PERIOD 60
#define RECEIVE_UNIXTIME_PERIOD 240

//GSM TO CONFIGURE
#define GSM_MQTT_BROKER ""
//#define GSM_PASS ""
//#define GSM_USER ""
//#define GSM_APN "orange"
//#define GSM_PIN_CODE "0000"
```

Figure 14: Extrait du fichier de configuration

Sur le code ci-dessus, l'utilisateur va pouvoir changer le nom enregistré dans la base de donnée de la station QameleO. On peut également changer le temps entre chaque mesures, le temps entre chaque envoies des données et le temps entre chaque récupération de l'heure.

Il est important de choisir d'utiliser le "#define" pour créer ces variables car à la compilation, on va économiser de la mémoire et des performances plutôt que de stocker ces informations dans des variables globales.

7 Le capteur Waou

La deuxième étape du projet WAQATALI est le développement du capteur Waou. Ce capteur va nous permettre de mesurer l'humidité dans le sol et nous permettre d'assurer un arrosage optimal. J'ai travaillé en binôme sur ce projet avec Paul Gruet-Masson. Je me suis occupé du développement de l'informatique embarqué qui va permettre la prise de mesures et Paul s'est occupé du développement des capteurs et des matériaux les composants.

7.1 Le fonctionnement

Paul a donc créé les matériaux pour les capteurs qui sont au final constitué de deux électrodes inoxydables, de supports imprimés 3D en PLA, d'un électrolyte solide (matériau "éponge" dont la conductivité varie en fonction de l'eau) et de deux fils électriques allant vers l'électronique de mesure. L'électrolyte solide est un type de béton cellulaire, qui est un matériau poreux capable d'absorber une certaine quantité d'eau par capillarité. Pour réaliser une mesure, on réalise le montage ci-dessous.

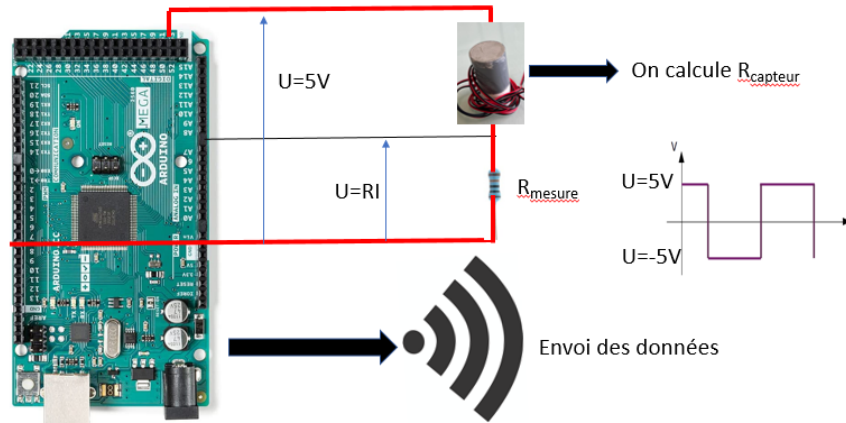


Figure 15: Schéma du montage pour réaliser une mesure

On commence par brancher en série notre capteur et une résistance. Il est important de connaître la valeur de cette résistance car elle influe sur la précision de la mesure. On va ensuite lire la tension entre le capteur et la résistance et grâce au principe du **pont diviseur de tension**, on va pouvoir calculer la résistance du capteur. Cependant, le capteur va réagir comme un condensateur, c'est à dire que les électrodes qui le composent vont se charger. Pour éviter cela, on va alterner le signal de mesure en le passant de 5V à -5V. Une fois que les mesures sont faites, on va pouvoir enregistrer ces données et les envoyées sur un serveur de la même façon que le QameleO.

7.2 Le capteur à mon arrivée

Pour tester ce pont diviseur de tension, le laboratoire a créé la boîte à "clic-clac" (voir figure ci-dessous).

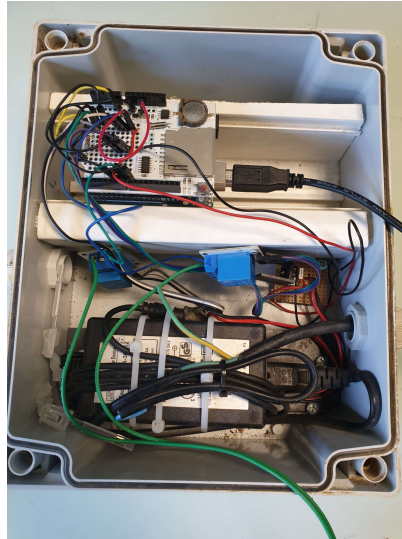


Figure 16: Photo de la boîte à "clic-clac"

C'est une boîte prototype qui va réaliser les mesures sur un capteur "test" et un capteur témoin, une sonde Watermark (voir figure ci-dessous). La sonde Watermark est capable également de mesurer l'humidité dans le sol. Cependant, on la trouve sur le marché à 38€ ce qui va à l'encontre du but "low cost" du projet WAQATALI.



Figure 17: Photo de la sonde Watermark

Pour effectuer les mesures sur les deux capteurs, le prototype utilise des relais. Ils vont permettre de transmettre le signal du circuit électrique au capteur sans contact galvanique entre les deux capteurs.

7.3 Développement du dispositif de mesure expérimental Waou

Nous avons la boîte à "clic-clac" nous permet de mesurer jusqu'à 4 capteurs. En laboratoire, on souhaite utiliser jusqu'à 8 capteurs. L'objectif de ce travail est de concevoir et développer un dispositif de mesure standardisé qui permettra de tester les différents prototypes de sonde Waou construite par Paul. Nous avons alors repris les principes développés dans la boîte à "clic-clac" que nous avons amélioré, testé et fiabilisé. Nous avons réalisé ce travail en plusieurs phases comme suit :

- Suppression des relais
- Choisir les composants et les branchements de la carte Waou
- Trouver les limites fréquentielle d'une carte Arduino

Dans la suite de cette section, nous développons chacune de ces phases.

7.3.1 Phase 1 : Suppression des relais

Il nous a donc fallu imaginer un système permettant de les brancher sur notre carte Arduino tout en évitant de complexifier notre circuit électrique. On va donc créer une carte électronique que l'on pourra brancher sur notre carte Arduino qui aura pour effet de lier les capteurs, les résistances et les pins sans fils auxiliaires.

Pour cela, il va falloir soit mettre des optocoupleurs, soit augmenter l'impédance pour éviter les perturbations lors des mesures. Il faut donc tester si la carte Arduino permet d'isoler toutes les sorties de la carte lorsqu'elle envoie une tension vers un capteur. Dans ce cas là, on dira qu'il s'agit d'une impédance forte. Pour tester l'impédance forte lors de la mesure d'une résistance, j'ai réalisé le montage ci-dessous :

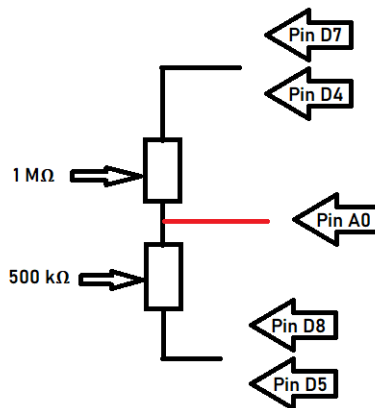
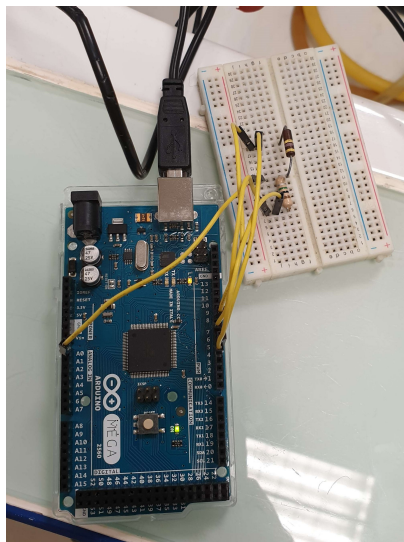


Figure 18: Montage électrique pour tester l'impédance

Pour effectuer le test, j'ai utilisé une résistance de mesure de $1\text{ M}\Omega$ (bloc jaune ci-dessus) et la tension lors de ces mesures est de 5 V . La résistance qui va remplacer le capteur et qui sera la cible des mesures est une résistance d'environ $500\text{ k}\Omega$ (bloc vert ci-dessous).

J'ai commencé en mettant tout les pins (ici, 7-8 et 4-5) en OUTPUT. On a récupéré la tension dans A0 et grâce à la fonction "getHumidity" du code Waou, on a calculé la résistance.

```
16:36:24.532 -> 436.9590148925
16:36:24.564 -> 2.14
16:36:24.564 -> Valeur entre les pin 7-8 : 745611.81
16:36:32.713 -> 435.0620117187
16:36:32.713 -> 2.13
16:36:32.713 -> Valeur entre les pin 4-5 : 739979.50
```

Figure 19: Capture d'écran des résultats de la mesure de résistance avec tout les pins en OUTPUT

Comme on peut le voir sur la figure ci-dessus, que ce soit entre les pins 7-8 ou 4-5, les résultats sont mauvais puisque l'on obtient une résistance d'environ $740\text{ k}\Omega$

Nous avons ensuite fais la même mesure en mettant 4-5 en INPUT lorsque l'on mesurait entre 7-8 puis en mettant 7-8 en INPUT en remettant 4-5 en OUTPUT.

```

16:32:49.039 -> Valeur entre les pin 7-8 : 514729.62
16:32:57.194 -> 347.5899963378
16:32:57.194 -> 1.70
16:32:57.194 -> Valeur entre les pin 4-5 : 514635.56
16:33:07.355 -> 347.5710144042
16:33:07.355 -> 1.70
16:33:07.355 -> Valeur entre les pin 7-8 : 514592.87

```

Figure 20: Capture d'écran des résultats de la mesure de résistance avec les pins en OUTPUT - INPUT

D'après la figure ci-dessus, on constate que les valeurs de résistances calculées sont proche de la valeur cible de 500 k Ω

On peut en conclure que l'utilisation d'octocoupleur n'est pas indispensable puisque l'on peut brancher les capteurs à la carte et lors de la mesure d'un capteur, mettre les autres en INPUT. Nous avons donc une impédance forte.

7.3.2 Phase 2 : Les composants et branchements de la carte Waou

Après avoir confirmé la suppression des relais, j'ai travaillé avec Mme. Manel Rami sur la conception d'une carte électronique nous permettant de réaliser notre montage de pont diviseur de tension pour 8 capteurs. Cette carte devra également utilisé un module SD comme sur QameleO et également un module RTC (voir figure ci-dessous).



Figure 21: Module RTC

Ce module servira à avoir une date précise, même en cas de problème car ce module utilise une pile lithium. Par conséquent, si un accident a lieu (une panne de courant par exemple), l'horloge tournerait toujours grâce à cette pile. L'avantage de cette méthode est qu'elle est très peu énergivore.

Nous avons donc décidé d'utiliser les pins suivant pour les capteurs :

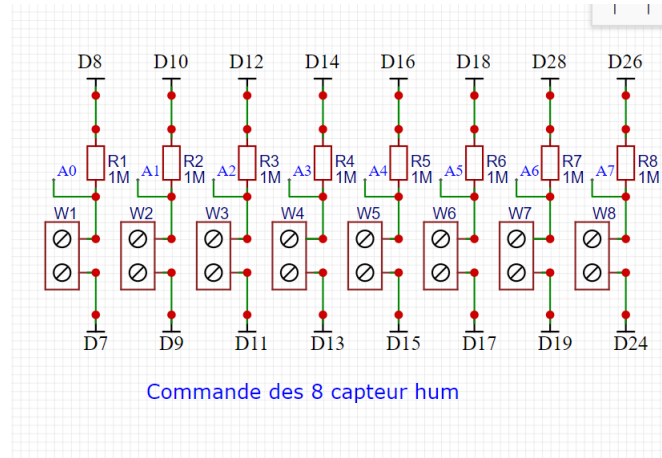


Figure 22: Schéma de la configuration de la carte électronique

Pour nos 8 capteurs, nous allons avoir besoin de 8 résistances et 8 borniers qui vont nous permettre de les brancher sur notre carte. Notre module SD sera branché sur les pins [53, 52, 51, 50, 5V, GND] et le module RTC sur [GND, 5V, 20, 21].

Après plusieurs semaines d'usinage, nous obtenons la carte électronique Waou (voir figure ci-dessous).

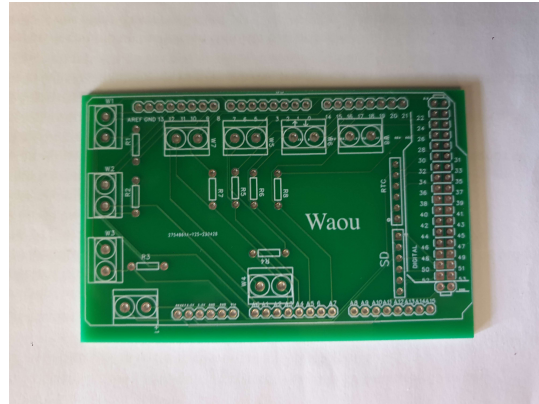


Figure 23: Photo de la carte Waou

Finalement, nous avons décidé d'utiliser des résistances de $100\text{ k}\Omega$ pour notre prototype. Grâce au test avec la sonde Watermark, on a pu avoir un ordre de grandeur des résistances à mesurer. La résistance du capteur va varier entre $100\text{ }\Omega$ et $500\text{ k}\Omega$. Nous avons remarqué que plus la valeur de la résistance du capteur était proche de la résistance connue, plus la précision était bonne. Il est donc préférable d'utiliser une résistance capable de mesurer avec précision les petites et les grandes valeurs.

J'ai soudé ensuite chaque bornier et résistance sur cette carte ainsi que les pins pour connecter la carte Waou à l'Arduino (voir figure ci-dessous).

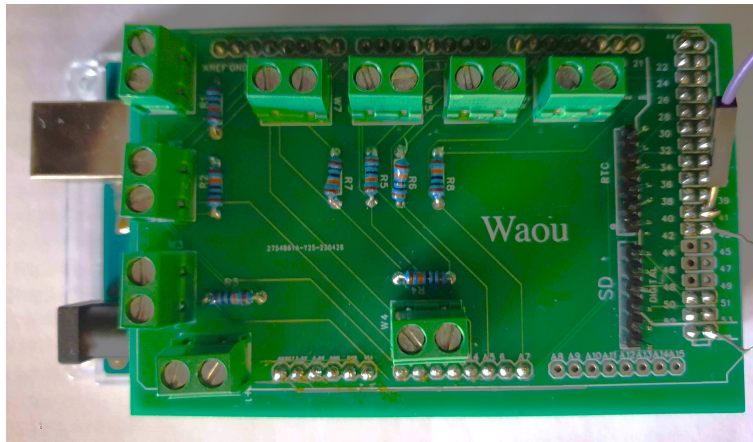


Figure 24: Photo de la carte Waou soudé

Il ne reste qu'à brancher les derniers composants et les capteurs pour avoir notre prototype Waou complet.

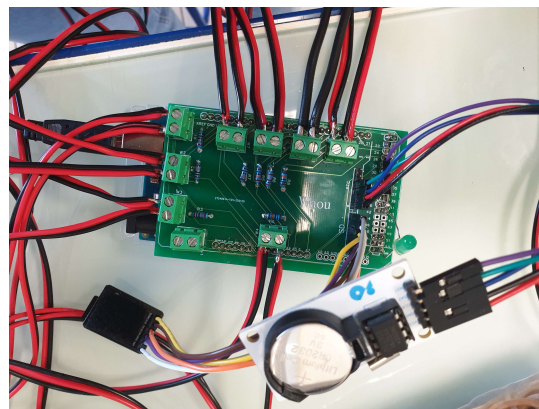


Figure 25: Prototype complet Waou

La carte est maintenant prête à l'emploi mais il reste à développer le code de l'informatique embarquée. Nous avons la fonction "getHumidity" déjà existante et qui est le coeur de ce projet. Le but de ce code est qu'il soit facile à utiliser par n'importe quel utilisateurs. L'utilisateur n'aura besoin de modifier que les lignes ci-dessous :


```

//Modifier le nombre de capteur à votre convenance
#define NB_SENSOR 8
//Temps entre 2 mesures en min
#define MEASURE_PERIOD 10
// On entre ici la fréquence en Hz souhaitée en sortie
#define frequenceDeSortie 500

```

Figure 26: Ligne de réglage du code Waou

Le code pourra alors à l'utilisateur de choisir le nombre de capteur qu'il souhaite brancher sur la carte (jusqu'à un nombre maximale de 8), choisir le temps entre chaque mesure car faire des mesures toutes les minutes n'est pas forcément le plus pertinent et est plus énergivore. Il pourra également choisir la fréquence des mesures. C'est à dire la vitesse à laquelle les 500 acquisitions vont se faire.

7.3.3 Phase 3 : Les limites fréquentielles d'une carte Arduino

Lors de nos mesures, on va faire 500 acquisitions par capteur. Cependant, ces acquisitions vont se faire à une certaine fréquence. On va donc tester les limites fréquentielles de la carte Arduino pour montrer l'évolution de la fréquence afin de voir le potentiel réel de notre carte Arduino Mega 2560 en réglant notre Arduino comme un GBF (Générateur Basse Fréquence). Ce test sera basé sur l'article suivant : <https://passionelectronique.fr/generateur-signal-carre-arduino-v1/>

Nous allons donc reprendre étape par étape les instructions de l'article et voir les conséquences avec une carte Arduino Mega 2560.

Tout d'abord, nous utilisons la méthode "digitalWrite" afin de passer la tension de 0V à 5V. La fonction "digitalWrite" est la fonction la plus commune pour modifier l'état d'un pin.

```

void setup() {
  pinMode(7, OUTPUT);
}

void loop() {
  digitalWrite(7, HIGH); //Passage à 5V
  digitalWrite(7, LOW); //Passage à 0V
}

```

Figure 27: Code du passage de 0V à 5V avec digitalWrite()

Après mesure sur oscilloscope, nous avons une fréquence de **86.96 kHz** (voir figure 35 en annexe).

Nous allons maintenant essayer de manipuler directement les ports de l'Arduino afin de changer l'état de notre pin.

Dans notre code, nous utilisons le pin 7. Il faut donc chercher sur la map de la

carte Arduino Mega 2560 à quel port correspond le pin 7 et sur quel bit il est positionné.

La documentation indique que le pin digital 7 est égale à PH4. Ce qui veut dire que le pin est sur le port H et que pour modifier son état, il faut toucher au 4ème bit.

```
#define MASK_PIN7 0b00010000

void setup() {
  DDRH = DDRH | MASK_PIN7; //Equivalent à pinMode(7, OUTPUT);
}

void loop() {
  PORTH = PORTH | MASK_PIN7; //Equivalent à digitalWrite(7, HIGH);
  PORTH = PORTH & ~MASK_PIN7; //Equivalent à digitalWrite(7, LOW);
}
```

Figure 28: Code du passage de 0V à 5V en modifiant l'état des pins

On crée un masque afin de modifier le pin 7. Ce masque est égale à **0b00010000**. On applique ensuite ce masque sur le port H (registre PORTH) afin de passer de 0V à 5V. Il faut également modifier le registre "DDRH" afin de préciser que le pin 7 est un OUTPUT ou un INPUT.

Avec le code ci-dessus, on mesure une fréquence de **1.14 MHz**. (voir figure 36 en annexe)

On explique cette forte augmentation de fréquence grâce à l'utilisation directe des pins. La méthode "digitalWrite" est une méthode complexe et remplie de vérification et d'appel à d'autres méthodes.

Afin de pouvoir limiter et définir la fréquence que l'on souhaite, nous allons devoir mettre des pauses dans le code. Nous utiliserons la fonction "_delay_us" pour cela car elle permet de faire des pauses de quelques nanosecondes.

```
void loop() {
  PORTH = PORTH | MASK_PIN7;
  _delay_us(0.85);
  PORTH = PORTH & ~MASK_PIN7;
  _delay_us(0.6);
}
```

Figure 29: Code de l'ajout d'un delay arbitraire

Les valeurs entrées dans la fonction ont été mises de façon arbitraire, sans raison particulière. Avec ces valeurs, nous obtenons une fréquence de **421 kHz** (voir figure 37 en annexe).

Maintenant, il faut modifier ce délai pour atteindre la fréquence souhaitée. Nous devons déjà calculer la période.

$$Periode = \frac{1}{421000} = 2.38 \mu s$$

Nous avons donc une période de 2.38 μs que l'on divise par 2 afin d'avoir le temps à un seul état (haut ou bas).

$$\frac{2.38}{2} = 1.19 \mu s$$

Ensuite, pour avoir le temps d'exécution à chaque état, on va soustraire le temps de base au temps d'un état. x_1 étant le temps d'exécution à l'état haut et x_2 à l'état bas.

$$x_1 = 1.19 - 0.85 = 0.34 \mu s$$

$$x_2 = 1.19 - 0.6 = 0.59 \mu s$$

Dans le code, nous aurons donc :

```
#define MASK_PIN7 0b00010000

#define frequenceDeSortie 20000 // On entre ici la fréquence souhaitée en
sortie (20000 Hz par défaut)
#define periodeDuSignal (float)1/frequenceDeSortie*1000000 // Période = 1/
Fréquence (et multiplié par 1 million, pour l'exprimer en microsecondes)
#define tempo (float)periodeDuSignal/2

void setup() {
    DDRH = DDRH | MASK_PIN7; //Equivalent à pinMode(7, OUTPUT);
}

void loop() {
    PORTH = PORTH | MASK_PIN7;
    _delay_us(tempo - 0.34);
    PORTH = PORTH & ~MASK_PIN7;
    _delay_us(tempo - 0.59);
}
```

Figure 30: Code de l'ajout d'un delay calculé

Maintenant, nous avons un code capable de régler la fréquence (voir figure 38 en annexe).

Nous allons faire plusieurs mesures en réglant la fréquence dans le code.

Fréquence théorique (code)	Fréquence mesuré (oscilloscope)
1 Hz	994.1 mHz
100 Hz	99.34 Hz
1 kHz	994 Hz
10 kHz	10.000 kHz
20 kHz	20.000 kHz
100 kHz	100.000 kHz
500 kHz	500.000 kHz
510 kHz	500.000 kHz
520 kHz	533.3 kHz
600 kHz	615.5 kHz
1 MHz	842.000 kHz

On constate que la fréquence est effectivement réglée correctement. Néanmoins, on remarque une limite à 500 kHz. Au-delà, la fréquence n'arrive pas à suivre réellement. On voit également que le signal n'est plus carré lorsque la fréquence est élevée (voir figure 39 en annexe) contrairement au signal à 20 kHz (voir figure 38 en annexe).

7.4 Réglage de la fréquence de mesure

Suite à l'étude du GBF, nous devons réussir à faire la même chose avec la fonction "getHumidity". Le principal problème est que la fonction "analogRead" limite fortement la fréquence et étant donné que nous utilisons deux pins afin de passer de 5V à -5V, la méthode utilisée pour le GBF sera modifiée.

7.4.1 Méthodes

En appliquant la méthode du GBF avec les mêmes calculs de fréquence et en indiquant une fréquence souhaitée de 100 Hz, nous obtenons une fréquence de 97.36 Hz. Le résultat semble plutôt correct.

Cependant, en souhaitant une fréquence théorique de 500 Hz, nous obtenons une fréquence de 446.3 Hz. **Les résultats ne sont donc pas bon.**

Pour tenter de résoudre ce problème, nous sommes partis du principe que si pour 100 Hz, nous obtenons 97 Hz alors il nous manque 3% de la valeur théorique. Donc lors du calcul de la période souhaitée, nous allons additionner à la fréquence 3% de celle-ci.

$$Periode = \frac{1}{FREQ_VOULU + (FREQ_VOULU * \frac{3}{100})}$$

Grâce à cette formule, nous obtenons bien une fréquence de 100 Hz lors de la pratique. Cependant cette formule ne fonctionne toujours pas pour une fréquence de 500 Hz. Il faut donc que le pourcentage ajouté soit également variable en fonction de la fréquence.

C'est pourquoi nous sommes arrivés à la formule suivante :

$$Periode = \frac{1}{FREQ_VOULU + (FREQ_VOULU * \frac{3 * \frac{FREQ_VOULU}{100}}{100})}$$

7.4.2 Résultat

Avec cette formule, nous obtenons les résultats suivants :

Fréquence théorique	Fréquence pratique
10 Hz	9.915 Hz
100 Hz	99.93 Hz
200 Hz	200.24 Hz
300 Hz	300.6 Hz
400 Hz	399.7 Hz
500 Hz	499.9 Hz
600 Hz	599.6 Hz
700 Hz	698.5 Hz
800 Hz	800.3 Hz
900 Hz	893.1 Hz
1 kHz	976.6 Hz

On voit que la fréquence arrive maintenant à être correctement ajustée. Cependant, cette méthode perd en précision lorsque la fréquence souhaitée augmente. **Il serait donc déconseillé de dépasser 1 kHz.**

7.5 Le capteur aujourd'hui

Actuellement, la carte Arduino est capable de mesurer la résistance de 8 capteurs et d'enregistrer ces mesures sur une carte SD. Le code est développé pour être le plus facile à prendre en main par des personnes sans connaissances en programmation et également pour rendre les mesures réglables à la convenance de l'utilisateur. On se sert donc de la carte développée pour tester les capteurs créés afin que les résultats soient les plus cohérents possible.

7.5.1 Résultats

Les capteurs créés par Paul ont été posés dans une table à succion. Cette table va servir à reproduire le pouvoir d'aspiration des plantes à différentes forces. On commence par imbibé d'eau nos capteurs puis en activant la table à succion, l'eau présent dans les capteurs va commencer à disparaître. C'est à dire que les capteurs vont sécher. En théorie, lorsqu'un capteur est plein d'eau, sa résistance est très faible (de l'ordre de la centaine de ohm). Plus le capteur perd en eau, plus la conductivité faiblit et donc, la résistance augmente. Cependant, nous avons les résultats suivants :

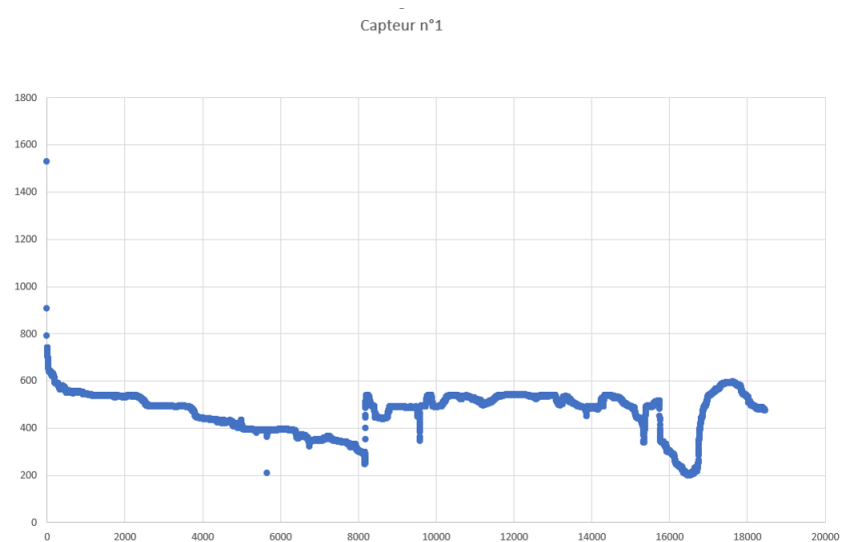


Figure 31: Résultat des mesures du capteur 1

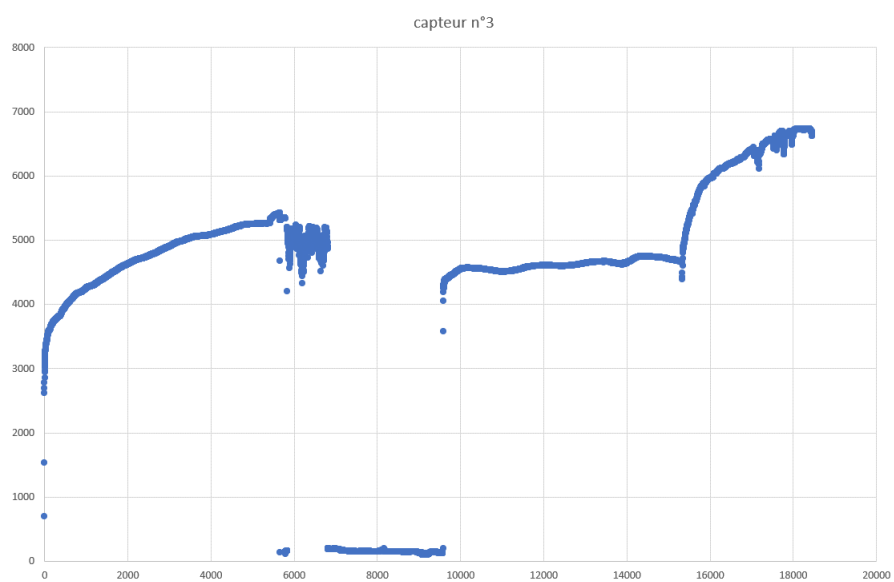


Figure 32: Résultat des mesures du capteur 3

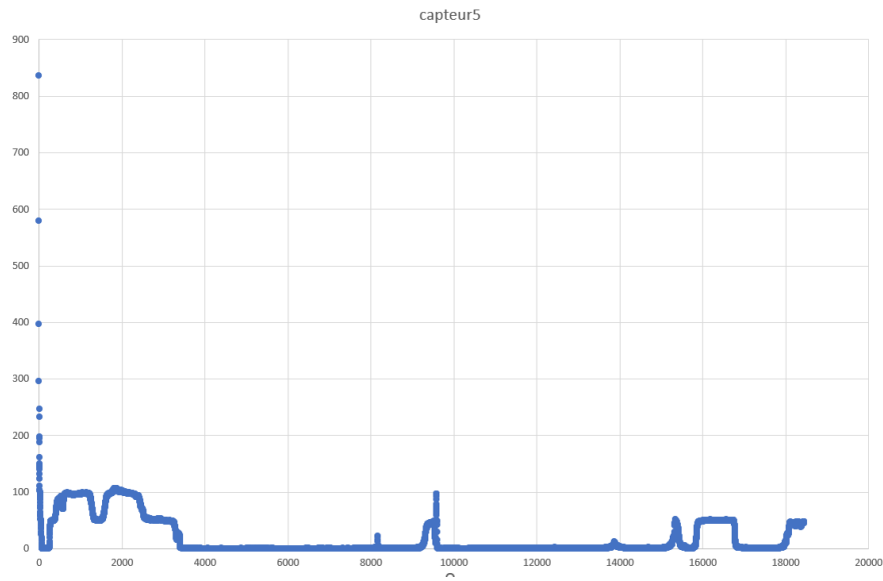


Figure 33: Résultat des mesures du capteur 5

Comme on peut le voir, ces graphiques sont très différents. La résistance du capteur 1 a tendance à baisser ce qui va à l'encontre de ce à quoi l'on s'attendait. La résistance du capteur 3 augmente, ce qui est rassurant. Le capteur 5, quant à lui, a une résistance extrêmement faible ce qui indiquerait un problème interne au capteur. On peut en déduire que nous avons un problème dans notre système.

7.5.2 Mise en place d'un protocole pour trouver la source de l'erreur

Nous avons donc des résultats très différents selon les capteurs. Nous avons donc effectué différents tests, que ce soit sur les capteurs ou l'informatique de mesure.

Nous avons commencé par tester chaque pin de mesure avec des résistances connues. Les résultats sont correctes, les résistances que l'on souhaite mesurer sont bien calculées par la carte. Après avoir mesuré chaque capteur avec un multimètre, nous avons remarqué des valeurs étranges probablement dû à l'effet capacitif de nos capteurs. Afin d'être sûr d'avoir les bonnes mesures de capteur, nous les avons testés sur un analyseur d'impédance. Cet appareil est capable de mesurer la résistance de nos capteurs en réalisant un pont diviseur de tension tout comme notre carte Waou. Après avoir comparé les résultats de l'analyseur d'impédance et les mesures faites par notre appareil, on a pu constater que les résultats étaient les mêmes ce qui indique que l'informatique embarquée réalise bien les bonnes mesures. Nous avons en plus testé les fréquences lors des mesures avec un oscilloscope et les résultats étaient également correctes. On

peut en déduire que le problème ne vient pas de l'appareil de mesure.

Une possibilité serait que les capteurs soient défectueux ou que les câbles reliant les capteurs à la carte Waou aient un problème. C'est pourquoi nous avons testé un capteur en utilisant un capteur sec. En théorie, nous sommes supposés mesurer une résistance d'au moins $1\text{ M}\Omega$.

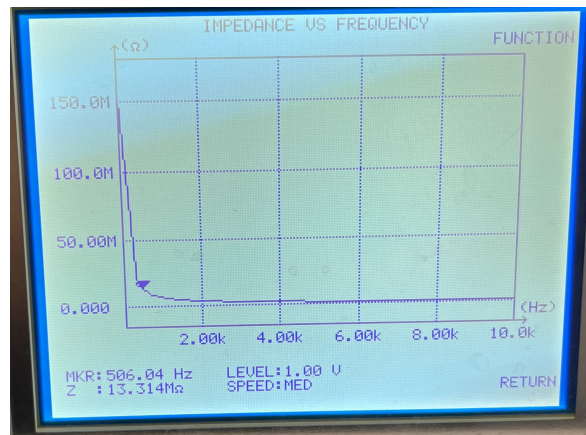


Figure 34: Graphique d'évolution de la résistance en fonction de la fréquence de mesure

La figure ci-dessus nous montre que lorsqu'on mesure avec une fréquence de 500 Hz, nous sommes supposés avoir une résistance de 13,314 M Ω . On peut déduire de ce résultat que le capteur n'a pas de problème puisque nous avons une valeur cohérente avec celle attendu.

On constate cependant que la fréquence de mesure a une forte influence sur le résultat. Diminuer ou augmenter cette valeur pourrait être une solution à notre problème. Il faut également tester les câbles dans le cas où ils seraient mal connectés aux électrodes du capteur.

8 Conclusion

8.1 Conclusion techniques

8.1.1 QameleO

L'objectif de ce stage sur la station environnemental QameleO était de corriger des bugs logiciels et d'écrire un tutoriel d'installation du code afin que tout utilisateurs potentiels puissent installer la station sans difficultés.

La station QameleO est maintenant capable d'envoyer les mesures sur un serveur avec les bonnes dates de mesure et la documentation d'installation du code est disponible en français et en anglais.

8.1.2 Waou

Concernant la station Waou, je devais concevoir et développer un dispositif de mesure standardisé qui permettra de tester les différents prototypes de sonde Waou construite par Paul.

Nous avons donc créer une carte électronique sur laquelle on est capable de brancher et mesurer 8 capteurs. Les capteurs sont toujours en phase de test mais l'informatique embarquée est opérationnelle pour le prototype. Cependant, il serait intéressant de pouvoir envoyer les données sur un serveur à l'instar du QameleO et pouvoir afficher les données sur une application mobile. Il serait également intéressant de réaliser des mesures capacitatives des capteurs. Etant donné que les capteurs sont biodégradables, faire une mesure capacitive d'un capteur permettrait d'avoir un bilan de santé du capteur et voir s'il est encore fonctionnel.

8.2 Conclusion personnel

D'un point de vue personnel, ce stage m'a appris à utiliser une démarche scientifique. C'est à dire qu'en cas d'erreur ou de réussite, je vais chercher pourquoi cela fonctionne ou non et faire en sorte de comprendre la source du problème (s'il y en a un). J'ai aussi appris les bases de l'informatique embarquée, la soudure de composants électroniques et à être rigoureux. J'ai également appris à communiquer avec des personnes venant d'autre milieux que moi et coopérer sur un projet commun et à prendre la parole lors d'évènement puisque j'ai participé à "Pint of Science" qui est un évènement de vulgarisation scientifique où Paul Gruet-Masson et moi avons présenté le capteur Waou et son fonctionnement.

9 Bibliographie

- Site de présentation du projet WAQATALI : <https://anr.fr/Projet-ANR-21-LCV3-0014>
- Site de l'IRD : <https://www.ird.fr>
- Site de présentation de l'UMMISCO : <https://www.ummisco.fr>
- site de UrbaSense : <https://www.urbasense.fr/fr/>
- Github de la station QameleO : <https://github.com/CoFab-in-Bondy/QameleO>
- Site des caractéristiques d'une carte Arduino Mega 2560 : <https://docs.arduino.cc/>
- Tutoriel pour la création d'un GBF : <https://passionelectronique.fr/generateur-signal-carre-arduino-v1/>
- Site de l'évènement "Pint of Science" : <https://pintofscience.fr/event/se-preparer-pour-lavenir>

10 Annexes

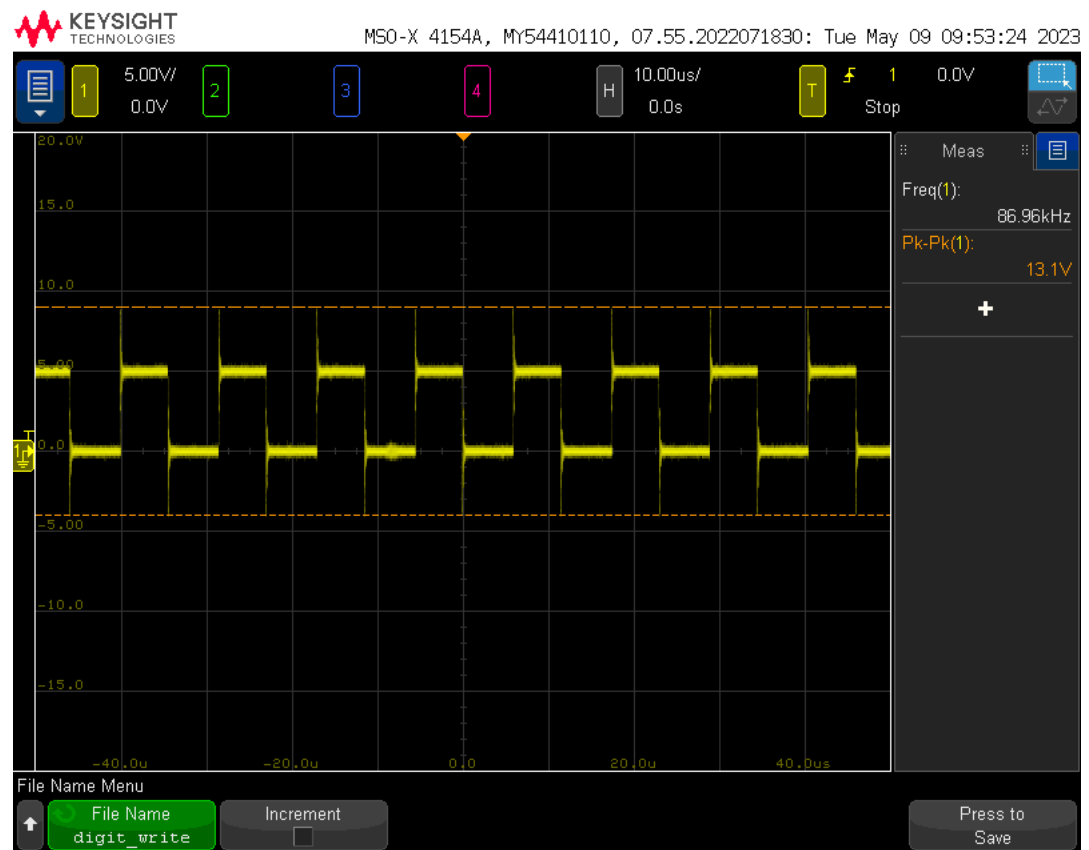


Figure 35: Mesure de la méthode "digitalWrite"

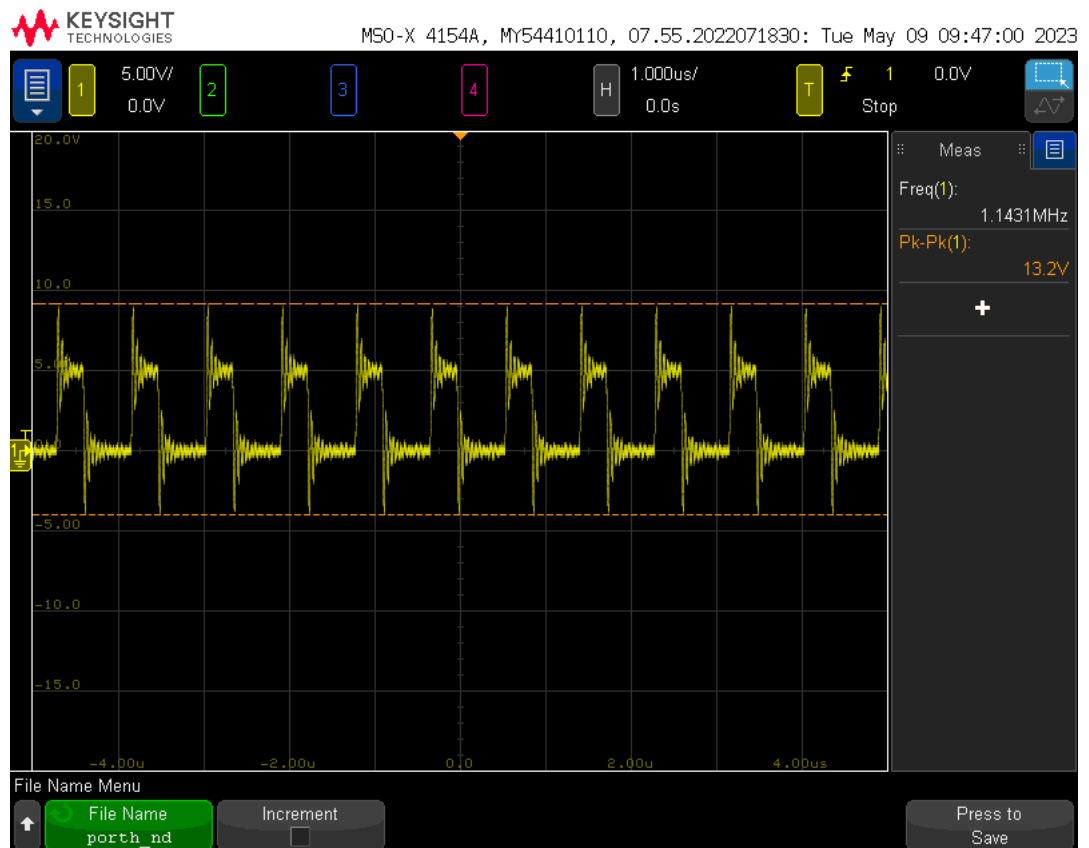


Figure 36: Mesure de la méthode "porth" sans délai

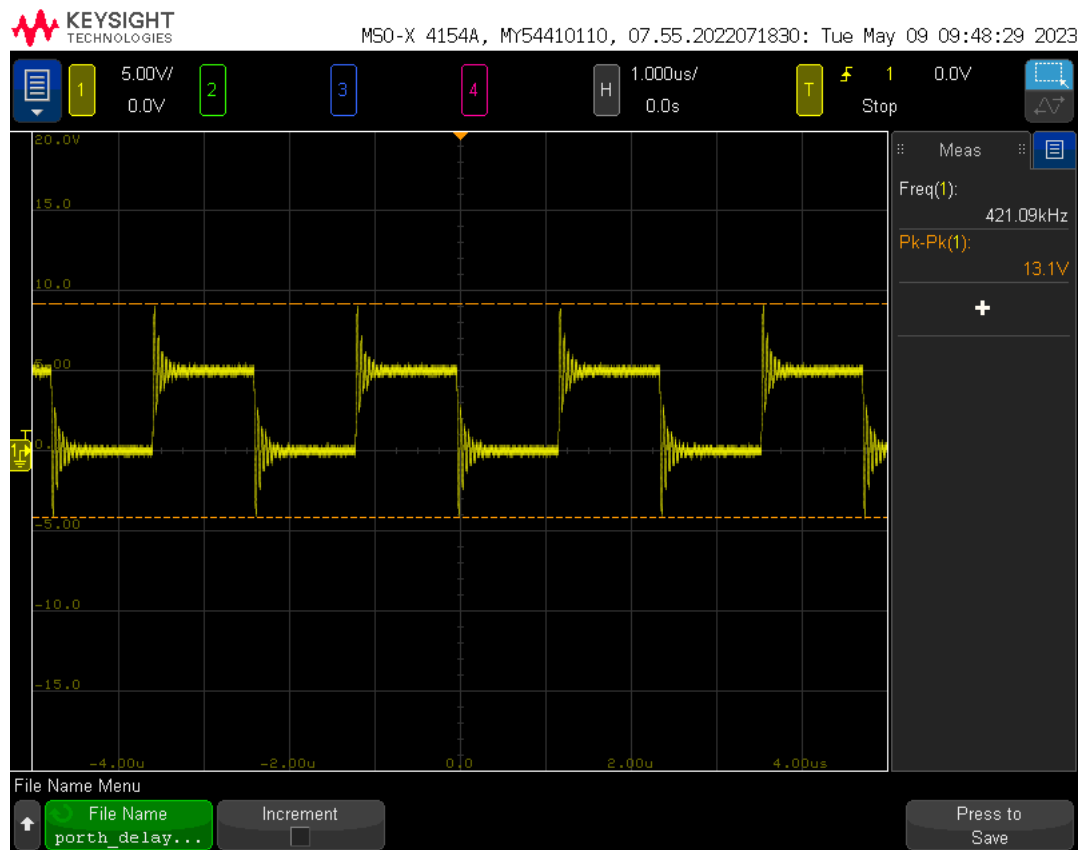


Figure 37: Mesure de la méthode "porth" avec délai par défaut

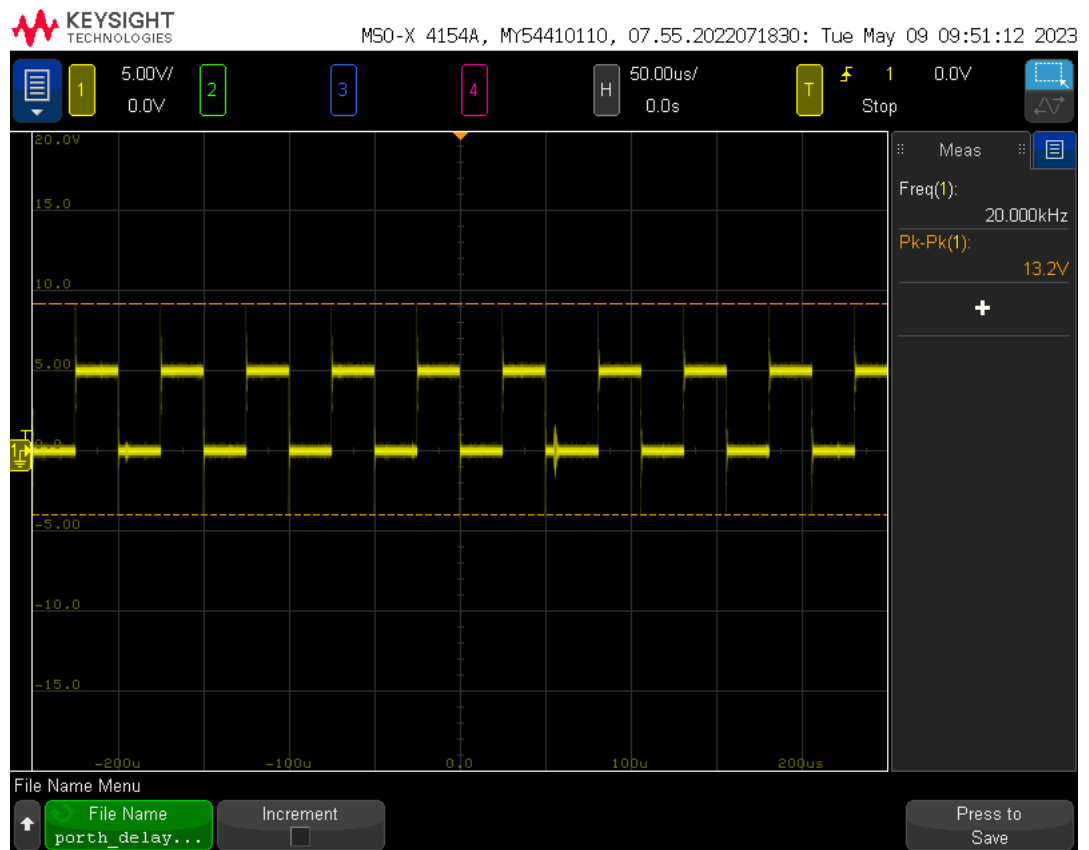


Figure 38: Mesure de la méthode "porth" avec délai réglé pour 20 kHz

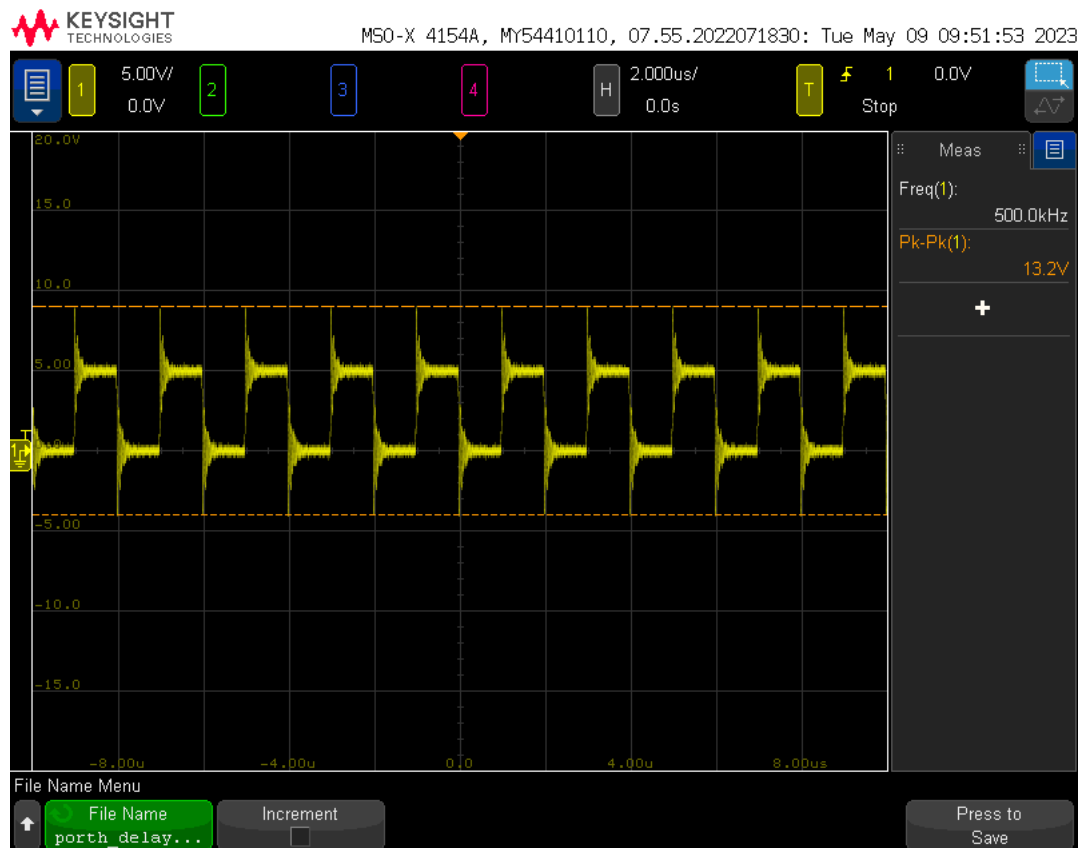


Figure 39: Mesure de la méthode "porth" avec délai réglé pour 500 kHz

11 Résumé / Abstract

Les villes et les sociétés actuelles connaissent de profondes mutations pour faire face aux enjeux climatiques, environnementaux et sociétaux qui sont prégnants au Nord et au Sud. C'est pourquoi le projet WAQATALI tente de répondre à ces questions. Le projet WAQATALI va utiliser plusieurs stations de mesures afin de répondre à ces problématiques. Ce rapport présente les fonctions de la station environnemental QameleO permettant de mesurer la qualité de l'air avec la correction des bugs logiciels rendant possible l'envoi des données à un serveur distant puis la conception et le développement du code de la station Waou, un appareil capable de déterminer l'humidité présente dans le sol par la mesure d'une résistance.

Mot-clés : Rapport, WAQATALI, Environnement, Informatique embarquée

Today's cities and societies are undergoing profound changes to in the face of significant climate, environmental and societal challenges in the North and South. This is why the WAQATALI project is trying to answers these questions. The WAQATALI project will use several measurement stations to address these issues. This report presents the functions of the QameleO environmental station to measure air quality with the correction of software bugs making it possible to send the data to a remote server then design and development of the code of the Waou station, a device capable of determining moisture in the soil by measuring resistance.

Keywords: Report, WAQATALI, Environment, Embedded IT